

BME 234 — Homework 3 Answers

Sage Clokey | Spring 2026

Problem 1: Random Forests (20 pts)

(a) Variance of the average of N uncorrelated variables (4 pts)

Given N uncorrelated variables $X_1, \dots, X_N$ each with mean μ and variance σ^2 :

The average is $\bar{X} = (1/N) \sum X_i$

Since the variables are uncorrelated, all covariance terms are zero:

$$\text{Var}(\bar{X}) = (1/N^2) \sum \text{Var}(X_i) = (1/N^2)(N\sigma^2) = \sigma^2/N$$

Averaging N uncorrelated classifiers reduces variance by a factor of N.

(b) Why 37% of samples are OOB for large M (4 pts)

When constructing a bootstrapped dataset of size M from an original dataset of size M (sampling with replacement), the probability that a specific data point is not selected in a single draw is $(1 - 1/M)$. Since we draw M times independently with replacement, the probability that a point is never selected is:

$$P(\text{OOB}) = (1 - 1/M)^M$$

Using the given hint:

$$\lim_{M \rightarrow \infty} (1 - 1/M)^M = 1/e \approx 0.3679 \approx \mathbf{37\%}$$

Therefore, for large M, approximately 37% of samples will not appear in any given bootstrapped dataset and are out-of-bag for that tree.

(c) OOB error and choosing the number of trees (2 pts)

The OOB error is computed by predicting each training sample using only the trees for which that sample was out-of-bag (i.e., not used during training). For each sample, predictions are aggregated only from trees that did not include it in their bootstrap sample, and these predictions are compared to the true label to compute an error rate.

To choose the number of trees: plot the OOB error as a function of the number of trees in the forest. As trees are added, the OOB error decreases and eventually stabilizes. The appropriate number of trees is where the OOB error plateaus — adding more trees

beyond this point provides diminishing returns. This serves as an alternative to cross-validation because the OOB samples act as a built-in validation set for each tree.

(d) Upper bound for variance with correlated variables (10 pts)

Given N correlated variables $X_1, \dots, X_N$, each with mean μ , variance σ^2 , and $0 \leq \text{Cov}(X_i, X_j) \leq \delta^2$ for all $i \neq j$:

$$\text{Var}(\bar{X}) = (1/N^2) \text{Var}(\sum X_i) = (1/N^2) [\sum \text{Var}(X_i) + \sum_{i \neq j} \text{Cov}(X_i, X_j)]$$

There are N variance terms and $N(N-1)$ covariance terms. Applying the upper bound $\text{Cov}(X_i, X_j) \leq \delta^2$:

$$\text{Var}(\bar{X}) \leq (1/N^2) [N\sigma^2 + N(N-1)\delta^2]$$

$$\text{Var}(\bar{X}) \leq \sigma^2/N + ((N-1)/N)\delta^2$$

As $N \rightarrow \infty$, the first term $\sigma^2/N \rightarrow 0$, but the second term approaches δ^2 . Unlike the uncorrelated case, variance cannot be reduced below δ^2 simply by adding more trees. This is why random forests decorrelate trees by using random feature subsets at each split — reducing δ^2 (the correlation between trees) is the only way to push the variance floor lower.

Problem 2: Neural Network Parameters (10 pts)

(a) Fully connected network (4 pts)

Each layer contributes (inputs $\times$ outputs) weights + outputs biases:

Connection	Weights	Biases	Total
Input (32) $\rightarrow$ Hidden 1 (128)	$32 \times 128 = 4,096$	128	4,224
Hidden 1 (128) $\rightarrow$ Hidden 2 (64)	$128 \times 64 = 8,192$	64	8,256
Hidden 2 (64) $\rightarrow$ Output (10)	$64 \times 10 = 640$	10	650

Total: $4,224 + 8,256 + 650 = 13,130$ parameters

(b) CNN parameters (6 pts)

Only convolutional layers have learnable parameters. ReLU activations and max pooling layers have no parameters.

Conv layer 1: 100 filters of size 3×3 (assuming single-channel input) - Each filter: $3 \times 3 \times 1 = 9$ weights + 1 bias = 10 parameters - 100 filters: $100 \times 10 = \mathbf{1,000}$ parameters

Conv layer 2: 50 filters of size 2×2 - Input to this layer has 100 channels (from conv layer 1's 100 filters) - Each filter: $2 \times 2 \times 100 = 400$ weights + 1 bias = 401 parameters - 50 filters: $50 \times 401 = \mathbf{20,050 \text{ parameters}}$

Total: $1,000 + 20,050 = \mathbf{21,050 \text{ parameters}}$

Note: ReLU and max pooling layers contribute 0 parameters.

Problem 3: Missense Variant Pathogenicity Prediction (50 pts)

Code submitted in `problem_3.py`.

(a) Dataset construction (4 pts)

After filtering the ClinVar VCF for variants with CLNSIG exactly matching "Benign" or "Pathogenic" (excluding "Likely_benign", "Likely_pathogenic", "Pathogenic/Likely_pathogenic", etc.):

- **Benign variants: 24,346** (label = 0)
- **Pathogenic variants: 20,596** (label = 1)
- **Total: 44,942 variants**

(b) Train/validation split (3 pts)

80/20 random split (random seed = 42):

Split	Benign	Pathogenic	Total
Training (80%)	19,439	16,514	35,953
Validation (20%)	4,907	4,082	8,989

(c) RVIS feature (7 pts)

RVIS (Residual Variation Intolerance Score) percentiles were assigned to each variant based on its gene. For genes without an RVIS score, a default percentile of 50 was used. For variants overlapping multiple genes, the average RVIS score was taken.

Distribution statistics:

Class	Mean RVIS	Median RVIS
Benign	54.64	54.24
Pathogenic	39.37	36.75

Histogram: See `rvis_histogram.png`.

Is RVIS useful? Yes. Pathogenic variants are concentrated in genes with lower RVIS percentiles (indicating less tolerance for functional variation), while benign variants are distributed more uniformly across percentiles. The ~15-percentile separation in means suggests meaningful discriminative power. Genes harboring pathogenic variants tend to be intolerant to missense variation, which is biologically expected — mutations in essential, constrained genes are more likely to cause disease.

(d) o/e feature (7 pts)

The o/e (observed/expected) loss-of-function upper rank was obtained from gnomAD v2.1.1 constraint data. For genes without an o/e score, the median rank was used as default. For variants overlapping multiple genes, the average was taken.

Distribution statistics:

Class	Mean o/e Rank	Median o/e Rank
Benign	7,518	7,532
Pathogenic	6,298	5,702

Histogram: See `oe_histogram.png`.

Is o/e useful? Yes. Pathogenic variants occur in genes with lower o/e ranks, meaning these genes are more constrained against loss-of-function mutations. Benign variants tend to be in genes with higher ranks (more tolerant). The separation in medians (~1,800 rank difference) provides discriminative signal. Like RVIS, o/e captures gene-level constraint, but it is computed from more recent data (gnomAD vs. ExAC) and uses more sophisticated statistical models.

(e) phastCons feature (7 pts)

phastCons scores were extracted from the `hg38.phastCons100way.bw` bigWig file using the `pyBigWig` library. Each variant's score reflects the probability that its genomic position is part of a conserved element across 100 vertebrate species.

Distribution statistics:

Class	Mean phastCons	Median phastCons
Benign	0.5501	0.8470
Pathogenic	0.9564	1.0000

Histogram: See `phastcons_histogram.png`.

Is phastCons useful? Yes. phastCons is expected to be a strong feature because pathogenic missense variants tend to occur at highly conserved positions — positions where evolution has maintained the same nucleotide across 100 vertebrate species because changes at these sites are deleterious. Benign variants tend to occur at less

conserved positions. Unlike RVIS and o/e, which are gene-level features, phastCons provides variant-level resolution, making it complementary to the gene-level scores.

(f) Random Forest training and evaluation (15 pts)

A Random Forest classifier was trained using three features: RVIS percentile, o/e rank, and phastCons score.

Hyperparameter selection: 5-fold stratified cross-validation was performed on the training set over the following grid: - n_estimators: [100, 200, 500] - max_depth: [5, 10, 20, None] - min_samples_leaf: [1, 5, 10]

Best hyperparameters:

Parameter	Value
n_estimators	500
max_depth	20
min_samples_leaf	1

Best cross-validation AUC: 0.9099

Validation set AUC: 0.9063

ROC curve: See roc_curve.png.

(g) Test set predictions (7 pts)

Predictions were generated for all variants in test_set.vcf using the trained Random Forest model. Each line of test_set.predictions contains the predicted probability of pathogenicity for the corresponding variant in the VCF file.